

CONVOLUTIONAL NEURAL NETWORKS

CNNs — Complete Study Notes

Colour-coded | Exam-ready | Examples included

► TOPICS COVERED

1. Fundamentals of CNNs
2. Convolution Operation Details
3. CNN Architecture and Layers
4. Pooling Layers and Techniques
5. Fully Connected (FC) Layer
6. Deep CNN Architectures
7. Data Augmentation in CNNs
8. Transfer Learning and Pre-trained Models

UNIT 1: FUNDAMENTALS OF CNNs

► What is a CNN?

A Convolutional Neural Network (CNN) is a deep learning model specially designed to process structured grid data like images. Unlike regular neural networks, CNNs exploit spatial relationships between pixels using local connectivity and weight sharing.

CNN vs Traditional Neural Networks

Feature	Traditional Neural Network	CNN
Input handling	Flattened 1D vector	2D / 3D spatial grid
Connectivity	Fully connected (all neurons)	Local connectivity (small region)
Weight sharing	No — each connection has unique weight	Yes — same filter slides across input
Parameters	Very large — not scalable for images	Much fewer — efficient
Spatial awareness	None	Preserved through conv layers
Best for	Tabular / structured data	Images, video, spatial data

Key Concepts

- ★ **Spatial Data**: Images are 2D grids of pixel values. A color image has shape $H \times W \times 3$ (height, width, RGB channels).
- ★ **Local Connectivity**: Each neuron connects only to a small local region of the input (called receptive field), not the entire image.
- ★ **Weight Sharing**: The same filter (kernel) is applied across ALL positions in the image. This drastically reduces parameters.
- ★ **Feature Extraction**: Filters detect patterns: edges, curves, textures in early layers; complex shapes in deeper layers.

Convolution Operation — How It Works

A small filter (kernel) slides over the input image. At each position, an element-wise multiplication followed by summation is performed (dot product). This produces a single number in the output feature map.

Step	Action	Example
1	Place kernel at top-left of input	3×3 kernel on 5×5 image
2	Multiply each kernel element with corresponding input pixel	9 multiplications
3	Sum all products → one output value	Sum = feature map cell
4	Slide kernel by stride S to next position	Stride = 1 → move right by 1
5	Repeat until entire image is covered	Output: feature map

Feature Maps

- **Output of applying a filter** to an input image is called a Feature Map
- Each filter (kernel) produces **one feature map**
- With K filters → K feature maps (K channels in output)
- Early layers: edges & simple patterns; Deeper layers: complex features

► **Worked Numerical Example — Convolution**

Input: 5×5 | Kernel: 3×3 | Stride: 1 | No Padding
 Output size = $(5 - 3 + 0) / 1 + 1 = 3 \rightarrow$ Output is 3×3
 Example: Input[0:3, 0:3] dot Kernel $\rightarrow$ output[0,0]
 If region = $\begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 1 \end{bmatrix}$ and kernel = $\begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix}$
 Output[0,0] = $(1 \times 1) + (0 \times 0) + (1 \times -1) + (0 \times 1) + (1 \times 0) + (0 \times -1) + (1 \times 1) + (0 \times 0) + (1 \times -1) = 0$



EXAM / NUMERICAL FOCUS

- ✓ Manual convolution calculation on a small matrix (very common!)
- ✓ Calculate output feature map size given input, filter, stride, padding
- ✓ Identify what each filter detects (edges, blobs, etc.)

UNIT 2: CONVOLUTION OPERATION DETAILS

► Key Parameters

Parameter	Symbol	Description
Input Size	$N \times N$ (square) or $H \times W$	Dimensions of input image/feature map
Filter Size	$F \times F$	Size of the kernel (e.g., 3×3 , 5×5)
Stride	S	How many pixels the kernel moves per step
Padding	P	Zeros added around the input border

► Output Size Formula

$$\text{Output Size} = \text{floor}((N - F + 2P) / S) + 1$$

Example: Input = 28×28 , Filter = 5×5 , Stride = 1, Padding = 0
Output = $(28 - 5 + 2 \times 0) / 1 + 1 = 23 + 1 = 24 \rightarrow$ Output feature map: 24×24

► Types of Convolution

1. Valid Convolution (No Padding)

- $P = 0$ (no padding added)
- Output is **smaller** than input
- Formula: $\text{Output} = (N - F) / S + 1$

2. Same Convolution (Zero Padding)

- Padding added so **output size = input size** (when $S=1$)
- Required padding: $P = (F - 1) / 2$
- Useful to preserve spatial dimensions through layers

► Effect of Stride and Padding

Change	Effect on Output Size
Increase Stride ($S \uparrow$)	Output size decreases (downsampling)
Increase Padding ($P \uparrow$)	Output size increases
Increase Filter Size ($F \uparrow$)	Output size decreases
$S=1$, Same padding	Output size = Input size



EXAM / NUMERICAL FOCUS

- ✓ Given $N, F, S, P \rightarrow$ compute output dimension

- ✓ How many multiplications are in a convolution? = $F \times F \times \text{output_H} \times \text{output_W}$
- ✓ Choose correct padding type for a given task
- ✓ Compute output size for non-square inputs (H × W format)

UNIT 3: CNN ARCHITECTURE AND LAYERS

► Full CNN Pipeline (Flowchart)

Stage	Layer	Purpose
Input	Image ($H \times W \times C$)	Raw pixel values
↓	Convolution Layer	Extract local features using filters
↓	Activation (ReLU)	Add non-linearity
↓	Pooling Layer	Reduce spatial size, retain important info
↓	Repeat Conv+Pool	Extract higher-level features
↓	Flatten	Convert 3D feature maps to 1D vector
↓	Fully Connected (FC)	Learn global patterns
Output	Softmax / Sigmoid	Final class probabilities

► Layer-by-Layer Details

Convolution Layer

- Applies K filters of size $F \times F$ to the input
- **Output volume:** $H_{\text{out}} \times W_{\text{out}} \times K$ (K = number of filters)
- Parameters per filter: $F \times F \times C_{\text{in}} + 1$ (bias)
- Total parameters: $K \times (F \times F \times C_{\text{in}} + 1)$

Activation Layer — ReLU

$$\text{ReLU}(x) = \max(0, x)$$

- Replaces all **negative values with 0**, keeps positive values unchanged
- Introduces non-linearity (without it, CNN = linear model)
- Most widely used; faster training, no vanishing gradient issue

Function	Formula	Use case
ReLU	$\max(0, x)$	Most layers — default choice
Sigmoid	$1 / (1 + e^{(-x)})$	Binary classification output
Tanh	$(e^x - e^{(-x)}) / (e^x + e^{(-x)})$	Older architectures
Softmax	$e^{x_i} / \sum e^{x_j}$	Multi-class output layer

► Parameter Count Formula

$$\text{Conv Layer Params} = K \times (F \times F \times C_{\text{in}} + 1)$$

Example: Conv layer with 32 filters of 3×3, input has 3 channels
 $\text{Params} = 32 \times (3 \times 3 \times 3 + 1) = 32 \times 28 = 896$ parameters



EXAM / NUMERICAL FOCUS

- ✓ Layer-wise dimension flow for a given CNN (step-by-step)
- ✓ Parameter count in each layer (especially FC layer)
- ✓ ReLU output for a given activation map
- ✓ Receptive field calculation

UNIT 4: POOLING LAYERS AND TECHNIQUES

► Purpose of Pooling

- **Reduce spatial dimensions** ($H \times W$) while keeping depth (channels) same
- Reduce number of parameters and computation
- Provide **spatial invariance** — small shifts don't change output
- Help **prevent overfitting**

► Types of Pooling

Type	Operation	Use Case
Max Pooling	Take maximum value from pool window	Most common — preserves strongest features
Average Pooling	Take average of all values in window	Smooth representations
Global Avg Pooling	Average entire feature map → single value	Replace FC layer in modern nets (GAP)
Global Max Pooling	Max over entire feature map	Rarely used directly

► Pooling Output Size Formula

$$\text{Pool Output} = \text{floor}((N - P_size) / S) + 1$$

Example: Input = 6×6 , Pool size = 2×2 , Stride = 2
Output = $(6 - 2) / 2 + 1 = 3 \rightarrow$ Output: 3×3 (size halved!)

► Manual Max Pooling Example

Input (4×4 region):

1	3	2	4
5	6	1	2
3	2	9	8
7	1	4	3

Pool size = 2×2 , Stride = 2
Top-left $2 \times 2 \rightarrow \max(1, 3, 5, 6) = 6$ | Top-right $2 \times 2 \rightarrow \max(2, 4, 1, 2) = 4$
Bot-left $2 \times 2 \rightarrow \max(3, 2, 7, 1) = 7$ | Bot-right $2 \times 2 \rightarrow \max(9, 8, 4, 3) = 9$
Output: $[[6, 4], [7, 9]]$



EXAM / NUMERICAL FOCUS

- ✓ Perform manual max pooling on a given matrix
- ✓ Compute output size after pooling
- ✓ Explain difference between max and average pooling
- ✓ Why is global average pooling used in modern architectures?

UNIT 5: FULLY CONNECTED (FC) LAYER

► Concept

- After all Conv + Pool layers, the **3D feature map is flattened** into a 1D vector
- This vector is fed into **dense (FC) layers** — each neuron connects to all inputs
- FC layers **learn global patterns** and perform the final classification

► Flattening Step

$$\text{Flatten size} = H \times W \times C \text{ (depth)}$$

Example: After all conv/pool layers, feature map = $7 \times 7 \times 64$ Flatten: $7 \times 7 \times 64 = 3136 \rightarrow$ becomes a vector of 3136 values

► FC Layer Parameters

$$\text{FC Layer Params} = (\text{inputs} \times \text{outputs}) + \text{outputs (bias)}$$

Example: Flatten $\rightarrow$ 3136 inputs, FC layer has 512 neurons
Parameters = $3136 \times 512 + 512 = 1,606,144 + 512 = 1,606,656$ ⚠ FC layers often dominate total parameter count! This is why modern nets use GAP instead.

Layer Type	Parameters Formula	Key Note
Conv Layer	$K \times (F \times F \times C_{in} + 1)$	Fewer params due to weight sharing
Pooling Layer	0	No learnable parameters!
FC Layer	$(in \times out) + out$	Usually largest parameter count
Output Layer (Softmax)	$(FC_out \times num_classes) + classes$	One neuron per class



EXAM / NUMERICAL FOCUS

- ✓ Calculate flattened dimension from a feature map
- ✓ Compute total parameters in an FC layer (very common exam question)
- ✓ Explain why FC layers have more params than Conv layers

UNIT 6: DEEP CNN ARCHITECTURES

► Architecture Comparison Table

Architecture	Year	Depth	Key Innovation	Use Case
LeNet-5	1998	7 layers	First successful CNN	Digit recognition (MNIST)
AlexNet	2012	8 layers	ReLU, Dropout, GPU training	ImageNet classification
VGGNet	2014	16–19 layers	All 3×3 filters, very deep	Feature extraction backbone
ResNet	2015	50–152 layers	Skip (residual) connections	Very deep nets, avoids grad loss
Inception (GoogLeNet)	2014	22 layers	Parallel filters (1×1,3×3,5×5)	Efficient computation

► LeNet-5

- Designed by Yann LeCun in 1998 for **handwritten digit recognition**
- Architecture: Input(32×32) → Conv1 → Pool1 → Conv2 → Pool2 → FC1 → FC2 → Output
- Used **sigmoid/tanh** activations (before ReLU era)

► AlexNet

- Won ImageNet 2012 challenge — sparked deep learning revolution
- Key innovations:
 - ReLU activation instead of sigmoid (faster training)
 - Dropout (0.5) to reduce overfitting
 - Data augmentation (flipping, cropping)
 - GPU-accelerated training

► VGGNet

- Very deep: 16 or 19 weight layers
- **Uses only 3×3 filters** throughout — key insight: two 3×3 convolutions = one 5×5 in terms of receptive field, but fewer parameters
- Simple, uniform architecture — great for transfer learning

► ResNet — Residual Networks

- **Skip connections** (shortcut connections) add input directly to the output of a block
- Solves the **vanishing gradient problem** in very deep networks
- A residual block computes: **$F(x) + x$** where $F(x)$ is learned transformation and x is the input

Why skip connections work: Without skip: gradient shrinks layer by layer (vanishing gradient). With skip: gradient flows directly from output to earlier layers → deeper nets can be trained!

► Inception Network (GoogLeNet)

- Uses **Inception modules**: multiple filter sizes (1×1 , 3×3 , 5×5) applied in parallel
- 1×1 convolutions used to **reduce dimensionality** cheaply before expensive larger convolutions
- Concatenates outputs of all parallel filters $\rightarrow$ richer feature maps
- Uses Global Average Pooling instead of large FC layers $\rightarrow$ far fewer params



EXAM / NUMERICAL FOCUS

- ✓ Compare architectures: depth, params, innovations
- ✓ Draw/describe a ResNet skip connection
- ✓ Why do deeper nets not always perform better? (vanishing gradient)
- ✓ Advantage of 3×3 filters vs larger filters

UNIT 7: DATA AUGMENTATION IN CNNs

► What is Data Augmentation?

Data Augmentation artificially increases the size of the training dataset by applying various transformations to existing images. This helps the model generalise better and prevents overfitting — especially when labelled training data is limited.

► Common Techniques

Technique	Description	Example Use
Horizontal Flip	Mirror image left-right	Cats, dogs — symmetrical subjects
Vertical Flip	Mirror image top-bottom	Satellite/aerial imagery
Rotation	Rotate image by random angle	Digits, medical scans
Random Crop	Crop a random portion of image	Object detection tasks
Color Jitter	Randomly change brightness/contrast/saturation	Lighting variation in real world
Zoom / Scaling	Zoom in or out randomly	Object at various distances
Noise Addition	Add Gaussian/salt-pepper noise	Robustness to sensor noise
Cutout / Masking	Randomly mask rectangular patches	Occlusion robustness

► Why Augmentation is Needed

- **Reduces overfitting**: Model sees more variation, generalizes better
- **Improves generalization**: Performance on unseen real-world images improves
- **Cost-effective**: Creates new training samples without collecting/labelling new data
- **Invariance**: Teaches model to recognize object regardless of position/orientation/lighting



EXAM / NUMERICAL FOCUS

- ✓ Identify which augmentation technique was applied (given image description)
- ✓ Explain why augmentation reduces overfitting
- ✓ List 4 augmentation techniques appropriate for a medical imaging task

UNIT 8: TRANSFER LEARNING & PRE-TRAINED MODELS

► What is Transfer Learning?

Transfer Learning uses a model already trained on a large dataset (e.g., ImageNet with 1M+ images) and applies its learned weights to a new, often smaller, task. Instead of training from scratch, you reuse learned features.

► Two Types of Transfer Learning

Type	What You Do	When to Use
Feature Extraction	Freeze all pre-trained layers; only train new FC head on top	Small dataset; task similar to original
Fine-Tuning	Unfreeze some/all layers; retrain entire or part of network with small learning rate	Larger dataset; task moderately different

► How Transfer Learning Works — Step by Step

Step	Action
1	Take a pre-trained model (e.g., VGG16, ResNet50 trained on ImageNet)
2	Remove the original output layer (1000-class softmax)
3	Add new layers for your task (e.g., 2-class softmax for cats vs dogs)
4	FREEZE early layers (weights not updated during training)
5	Train only the new top layers on your small dataset
6	Optional: Unfreeze some deeper layers and fine-tune with low learning rate

► Advantages of Transfer Learning

- **Less data required:** Pre-trained features work well even with small datasets
- **Faster training:** No need to train from scratch — converges quickly
- **Better performance:** Pre-trained models encode powerful low-level features (edges, textures)
- **Cost-effective:** Saves GPU time and compute resources

► Feature Extraction vs Fine-Tuning

Aspect	Feature Extraction	Fine-Tuning
Frozen layers	All pre-trained layers frozen	Some or all layers unfrozen

Trainable layers	Only new head (FC layers)	Partial or full network
Dataset needed	Small dataset OK	Larger dataset preferred
Training time	Very fast	Longer
Risk	May underfit if task very different	Risk of overfitting with tiny data
Learning rate	Normal	Very small (e.g., 1e-5)

► Common Pre-trained Models

Model	Trained On	Good For
VGG16 / VGG19	ImageNet	General image classification, feature extraction
ResNet50 / ResNet101	ImageNet	Deep feature extraction, fine-tuning
InceptionV3	ImageNet	Efficient feature extraction
MobileNet	ImageNet	Mobile / edge deployment (lightweight)
BERT / GPT (NLP)	Large text corpora	NLP transfer learning



EXAM / NUMERICAL FOCUS

- ✓ When should you use feature extraction vs fine-tuning?
- ✓ What is the role of frozen layers in transfer learning?
- ✓ How does transfer learning help with small datasets?
- ✓ Name 3 popular pre-trained CNN models and their source dataset
- ✓ Describe the fine-tuning process step by step

UNIT ★: QUICK REFERENCE — KEY FORMULAS & FACTS

Formula / Fact	Expression
Conv Output Size	$(N - F + 2P) / S + 1$
Same Padding Required	$P = (F - 1) / 2$ (for $S=1$)
Conv Layer Parameters	$K \times (F \times F \times C_{in} + 1)$
Pooling Output Size	$(N - Pool_size) / Stride + 1$
Flatten Size	$H \times W \times C$
FC Layer Parameters	$(inputs \times outputs) + outputs$
ReLU Activation	$\max(0, x)$
ResNet Block	$Output = F(x) + x$
Number of Conv Operations	$F \times F \times H_{out} \times W_{out} \times C_{in} \times K$